

2-D ARRAY

DEFINITION

Many applications require that data be organized in more than one dimension. One common example is a matrix (a table), which is an array that consists of rows and columns.

Syntax:

`type arrayname[size of dimension 1][size of dimension 2];`

Dimension 1 denotes rows

Dimension 2 denotes columns

E.g.

`int table [3][3];`

denotes a 2 dimensional array with 3 rows and 3 columns.

DECLARATION

Like one dimensional array, declaration of 2D array tells the compiler the name of the array, the type of its elements, and the size of each dimension.

E.g.1

```
Int table[4][4]={0,0,0,0,1,1,1,1,2,2,2,2,3,3,3,3};
```

E.g. 2

```
int table[4][4]={  
    {0,0,0,0},  
    {1,1,1,1},  
    {2,2,2,2},  
    {3,3,3,3}  
};
```

E.g.3

```
int table[][3]= {  
    {0,0,0},  
    {1,1,1}  
};
```

E.g.4

```
int table[2][3]={  
    {1,1},  
    {2}  
};
```

this will initialize first two elements of row one to 1, first element of row two to 2 and rest of the elements as 0.

E.g.5

```
Int arr[3][5]={ {0}, {0}, {0} };
```

This will initialize all the elements to zero.

INPUTTING VALUES

```
int table[3][4];
```

```
for(i=0;i<3;i++)
```

```
{
```

```
    for(j=0;j<4;j++)
```

```
    {
```

```
        scanf("%d",&table[i][j]);
```

```
    }
```

```
}
```

OUTPUTTING VALUES

```
for(i=0;i<3;i++)  
{  
    for(j=0;j<4;j++)  
    {  
        printf("%d",table[i][j]);  
    }  
}
```

MULTIDIMENSIONAL ARRAY

C allows arrays of three or more dimensions.

Syntax:

Type nameof array[d1][d2][d3]....[dn];

E.g.

```
int table [3][5][4];
```

EXAMPLE

```
int table[2][2][2]={  
    {  
        {1,1}  
        {1,1}  
    },  
    {  
        {2,2}  
        {2,2}  
    }  
};
```

Bit wise operators
